QA Agent — Code Deep-Dive

How the code actually works: the Python & Playwright mechanics, line by line

Companion to the Walkthrough. This document assumes you know *what* each stage does and now asks *how*.

> **Read this part first.** Five ideas show up in *every* file. Once these click, the rest of the code stops looking like magic. We cover them once here, then use them everywhere below.

Part A — The five foundations

1. `async` / `await` and why everything has it

Almost every function in this project is written `async def`, and almost every call has `await` in front of it. Here's the mental model.

A normal function *runs to completion* before giving control back. But talking to a browser or an API means **waiting** — for a page to load, for a network reply. While we wait, the CPU has nothing to do. Blocking the whole program during that wait is wasteful.

An `async def` function is a **coroutine**: a function that can *pause* itself at an `await` point, hand control back to a scheduler (the "event loop"), and resume later when the thing it was waiting for is ready.

```
async def goto(self, url, *, wait_until="domcontentloaded"):
    await self.page.goto(url, wait_until=wait_until)   # pause here until navigation finishes
    ...
```

- `await X` means: "start X, and pause me until X has a result." You can only use `await` *inside* an `async def`.
- Calling an async function **without** `await` does *not* run it — it just creates a coroutine object that sits there doing nothing. Forgetting `await` is the #1 async bug. (Remember the `match.count()` bug — a missing call/await silently returns the wrong thing.)
- The whole chain has to be async top to bottom. That's why `main()` is `async` and the very bottom of the script says `asyncio.run(main())` — that one line starts the event loop and runs the coroutine to completion.

> **Why it matters here:** reading a page calls `await` dozens of times (one per attribute, per element). Async lets those waits overlap with other work instead of freezing. You don't manage that overlap by hand — you just write `await` and the event loop handles the scheduling.

2. Playwright's two ways to point at an element: `ElementHandle` vs `Locator`

This is the single most important Playwright concept, and the code uses **both** deliberately.

	ElementHandle	Locator
Created by	`page.query_selector_all(...)`	`page.locator(...)`
What it is	A reference to **one specific DOM node, captured right now**	A **recipe** for finding element(s) — re-run every time you use it
If the page changes	Can go *stale* (node detached → errors)	Always re-queries the live page → never stale
Used in	The **Snapshotter** (read the page once)	The **Executor's _mui_select** (act on a live, changing popup)

The Snapshotter grabs handles because it's taking a *photograph* of the page at one instant:

```
handles = await page.query_selector_all("input, button, a, select, ...")
for handle in handles:
    tag = await handle.evaluate("el => el.tagName.toLowerCase()")  # this exact node
```

The Executor uses a Locator because the dropdown popup appears, changes, and disappears — a handle captured before the popup opened would be useless:

```
options = self.page.locator("[role='option']")   # a recipe, not a snapshot
count = await options.count()                      # evaluated NOW, against the open popup
await options.nth(target_idx).click()              # re-found NOW, then clicked
```

> **Why not handles everywhere?** Because after we click a dropdown, the DOM mutates. A locator says "find the 3rd `[role='option']` *at the moment I click*" — robust to that mutation. A stale handle would throw.

3. The Python ↔ browser bridge: `handle.evaluate("el => ...")`

This pattern is everywhere in the Snapshotter and it confuses everyone at first. Here's the truth:

The string inside `evaluate(...)` is JavaScript, and it runs *inside the browser*, not in Python. Python sends that JS source code over to Chromium, Chromium executes it against the real DOM, and sends back the *result* (which must be JSON-serializable — a string, number, list, etc., never a live DOM node).

```
mui_label = await handle.evaluate("""el => {
    const fc = el.closest('.MuiFormControl-root');   # runs in the browser
    if (!fc) return null;
    const lbl = fc.querySelector('label, .MuiInputLabel-root');
    return lbl ? lbl.innerText.trim() : null;        # a string comes back to Python
}""")
```

- `el` is the handle you called it on — Playwright injects your specific node as the argument.
- Inside, you have the full browser DOM API: `closest`, `querySelector`, `innerText`.
- The return value crosses back to Python. Here it's `"Doctor"` or `None`.

> **Why drop into JS at all?** Walking the DOM — "climb up to the form wrapper, find the label inside it" — is dozens of tiny operations. Doing each one as a separate `await` round-trip from Python would be slow and verbose. One `evaluate` does the whole walk *in the browser* and returns just the answer. It's one round-trip instead of twenty.

4. Pydantic models: typed data with automatic validation

Every `class X(BaseModel)` is a Pydantic model. Pydantic does three jobs for you:

```
class Element(BaseModel):
    tag: str                                # required, must be a string
    element_type: str | None = None         # optional, defaults to None
    widget_type: WidgetType = "native"      # must be one of the allowed Literals
    options: list[str] | None = None
```

1. **Validation on construction.** `Element(tag=123)` raises an error — `tag` must be a string. You can't build a malformed Element by accident.
2. **Defaults.** Fields with `= ...` are optional. That's why old code that never set `options` still worked — it defaulted to `None`.
3. **Literal types are enums.** `WidgetType = Literal["native", "mui_select"]` means the field can *only* be one of those two strings. Typo "muiselect"? Validation error. This is how the model documents and enforces the allowed widget kinds.

> **The mutable-default trap (and how this code avoids it).** In `SafetyVerdict` you'll see `signals: list[str] = Field(default_factory=list)` — *not* `= []`. Writing `= []` as a default in Python is a classic bug: *every* instance would share the *same* list object. `default_factory=list` tells

> Pydantic "call `list()` fresh for each new object." Remember this one — it bites every Python beginner.

Two Pydantic methods you'll see constantly:

- `obj.model_dump()` → turns the object into a plain `dict` (used in your print logs).
- `Schema.model_validate_json(raw)` → parses a JSON *string* and validates it into an object (used to turn the LLM's reply into a typed result — see Part B).

5. `Protocol` : the LLM provider is swappable by shape, not inheritance

`src/llm/base.py`

```
@runtime_checkable
class LLMProvider(Protocol):
    async def text(self, prompt: str, *, model=None, ...) -> str: ...
    async def structured(self, prompt: str, schema, *, ...) -> BaseModel: ...
```

A `Protocol` defines a *shape*: "anything with these two methods counts as an LLMProvider." `GroqProvider` does **not** inherit from `LLMProvider` — it just happens to have `text` and `structured` methods with the right signatures, so it qualifies. This is "structural typing" (a.k.a. duck typing, checked by the type checker). The payoff: to add an OpenAI provider later, you write a class with the same two methods and change one line in `get_provider()` — nothing else in the codebase changes, because everything depends on the *shape*, not the concrete class.

Part B — The LLM call, demystified

`src/llm/groq_provider.py`

Everywhere the code "asks the AI," it calls `llm.structured(prompt, SomeModel)` and gets back a validated object. How does free-form text become a guaranteed-valid Pydantic object? Here's the whole trick:

```
async def structured(self, prompt, schema, *, model=None, temperature=0.0, system=None):
    json_schema = schema.model_json_schema()           1
    sys_msg = (
        (system + "\n\n" if system else "")
        + "Respond with a single JSON object that matches this JSON schema. "
        + "Do not wrap it in markdown.\n"
        + json.dumps(json_schema)                       2
    )
    resp = await self._call(
        model=model or self._default_model,
        messages=[
            {"role": "system", "content": sys_msg},
            {"role": "user", "content": prompt},
        ],
        temperature=temperature,
        response_format={"type": "json_object"},        3
    )
    raw = resp.choices[0].message.content or "{}"        4
    try:
        return schema.model_validate_json(raw)           5
    except ValidationError:
        log.error("groq.structured.invalid_json", raw=raw)
        raise
```

1. **1** `schema.model_json_schema()` — Pydantic generates a **JSON Schema** (a machine-readable description) of the model. For `FormFill` it describes "an object with a `values` field that is a string→string map."
2. **2** That schema is jammed into the **system message** as instructions: "reply with JSON matching this shape." Now the model knows the exact structure expected.
3. **3** `response_format={"type": "json_object"}` is **JSON mode** — the API guarantees the reply is syntactically valid JSON (no prose, no markdown fences). This removes a whole class of parsing failures.
4. **4** Pull the text out of the API response envelope. `or "{}"` guards against a null content.
5. **5** `model_validate_json(raw)` — parse *and* validate in one step. If the JSON matches the schema, you get a typed object (`FormFill` , `SafetyVerdict` , …). If it doesn't, Pydantic raises `ValidationError` and we log the bad payload.

> **The big idea:** the LLM's job is reduced to "fill in this exact JSON shape," and Pydantic is the bouncer at the door that refuses anything malformed. Callers like `happy_path` or `SafetyGate`

> never see raw text — they get a clean, typed object or an exception. That's why the rest of the code can be so tidy.

Why `_call` wraps everything in a retry loop

```
for attempt in range(self._max_retries):
    try:
        return await self._client.chat.completions.create(**kwargs)
    except RateLimitError as e:
        delay = self._retry_after(e) or self._backoff(attempt)
        await asyncio.sleep(delay)        # wait, then loop again
    except (APIConnectionError, APITimeoutError, APIStatusError) as e:
        await asyncio.sleep(self._backoff(attempt))
```

APIs fail transiently — rate limits, blips. Rather than crash the whole test run, `_call` retries up to 5 times with **exponential backoff**: `_backoff` returns `base * 2^attempt` (1s, 2s, 4s, 8s…) capped at 30s, plus a little random "jitter" so many clients don't all retry in lockstep. For rate limits it first honors the server's `retry-after` header if present. This is the difference between a toy and something that survives a real run.

Part C — Reading the page, line by line

`src/browser/snapshotter.py — extract_elements()`

The query

```
handles = await page.query_selector_all(
    "input, button, a, select, textarea, "
    "[role='button'][aria-haspopup='listbox'], "
    "[role='combobox']"
)
```

Note the three adjacent strings with no commas between them — Python **implicitly concatenates** string literals that sit next to each other, so this is *one* selector string. (This is exactly the spot where an accidental comma once turned it into three arguments and crashed.) The result is a `list[ElementHandle]`.

The per-element loop — what each line extracts

```
for handle in handles:
    if await handle.get_attribute("aria-hidden") == "true":
        continue                                          1
    tag = await handle.evaluate("el => el.tagName.toLowerCase()")   2
    element_type = await handle.get_attribute("type")               3
    role = await handle.get_attribute("role")
    aria_haspopup = await handle.get_attribute("aria-haspopup")
    widget_type = "mui_select" if (
        aria_haspopup == "listbox" or role == "combobox"
    ) else "native"                                       4
    options  = await self._extract_options(handle, tag, widget_type)
    name     = await self._resolve_name(handle, tag)
    selector = await self._pick_selector(handle, tag)
    required = await handle.get_attribute("required") is not None    5
    visible  = await handle.is_visible()                  6
    disabled = await handle.is_disabled()
    kind     = await self._infer_kind(handle, tag, element_type, name)
    in_form  = await handle.evaluate("el => !!el.closest('form')")   7
    results.append(Element(...))
```

1. **1 Skip phantoms.** `get_attribute` returns a *string or* `None`. MUI scatters invisible helper inputs with `aria-hidden="true"`; `continue` skips to the next handle so we never fill those.
2. **2 Tag name via JS** because there's no direct handle property for it — one tiny `evaluate` returns `"div"`, `"input"`, etc.
3. **3** `type` is just an attribute (`"text"`, `"email"`, `"submit"`…). For a `<div>` it's `None`.
4. **4 The widget decision.** A Python ternary: if the element advertises a listbox popup or is a combobox, it's an MUI Select; otherwise native. This one boolean drives the whole fill-strategy later.

5. **5** `is not None` **turns presence into a bool.** A required field has the attribute present (value `""`); a non-required one returns `None`. We don't care about the value, only whether it exists — hence `is not None`.
6. **6** `is_visible()` / `is_disabled()` are Playwright **actionability** checks — they account for CSS (`display:none`, `visibility:hidden`, zero size). We store these so downstream code can skip things a user couldn't actually use.
7. **7** `!!el.closest('form')` — `closest('form')` climbs ancestors looking for a `<form>`; it returns the node or `null`. The `!!` in JS coerces that to a real `true` / `false`, which crosses back to Python as a bool. This is the `in_form` flag that powers `find_submit`.

_resolve_name — the label-finding ladder, in detail

It tries clue after clue and `return`s the moment one works (so order = priority):

```
mui_label = await handle.evaluate("""el => {
    const fc = el.closest('.MuiFormControl-root');
    if (!fc) return null;
    const lbl = fc.querySelector('label, .MuiInputLabel-root');
    return lbl ? lbl.innerText.trim() : null;
}""")
if mui_label:
    return mui_label                  # <- this rung fixed "Today" vs "Time"
```

MUI wraps each field in a `.MuiFormControl-root` div containing both the control and its label. We climb to that wrapper and read the label inside — so we get the field's *name* ("Time", "Doctor"), not its current *value* ("Today", "Dr. Vivek"). The remaining rungs (`aria-label` → `aria-labelledby` → `<label for>` → `placeholder` → button text → `"(unnamed)"`) are tried only if the one above returned nothing. Each rung handles a different way real apps label fields.

_infer_kind — a cascade from most to least reliable signal

```
if tag in ("button", "a"): return "unknown"             # not data fields
if tag == "input" and element_type in ("submit","reset","button","checkbox",...):
    return "unknown"
if element_type in type_map: return type_map[element_type]   # type="email" -> email
# then autocomplete attribute...
# then keyword-match label+id+placeholder against _KINDKEYWORDS
return "unknown"
```

The order is deliberate: a real HTML `type="email"` is rock-solid, so it's checked first; keyword-guessing the label is a last resort. One subtle line:

```
real_name = name if name != "(unnamed)" else ""
```

The sentinel `"(unnamed)"` literally contains the substring "name", which would falsely match the `name` keyword bucket. Blanking it out prevents that false positive — a small but real bug-fix baked into the code.

_pick_selector — generating a reliable address

Three cooperating methods. `_candidates` proposes selectors strongest-first:

```
data-testid  →  #id (if not autogenerated)  →  tag[name="..."]  →  a[href]  →  tag:text-is(".
```

`_pick_selector` returns the first candidate that is *unique*:

```
for candidate in await self._candidates(handle, tag):
    if await self._is_unique(candidate):    # query_selector_all(...) returns exactly 1?
        return candidate
return await self._positional_selector(handle)   # last resort
```

`_is_unique` simply runs the selector and checks `len(matches) == 1` — a selector that matches two elements is useless for "click *this* one." When nothing unique exists, `_positional_selector` builds a path in JS by walking parent-by-parent up to `<body>`, adding `:nth-of-type(n)` whenever siblings share a tag:

```
el => {
  const parts = [];
  while (el && el.tagName !== 'BODY') {
    let sel = el.tagName.toLowerCase();
    const sameTag = [...el.parentElement.children].filter(c => c.tagName === el.tagName);
    if (sameTag.length > 1) sel += `:nth-of-type(${sameTag.indexOf(el)+1})`;
    parts.unshift(sel);
    el = el.parentElement;
  }
  return parts.join(' > ');
}
```

That's the source of your `div > div > main > div:nth-of-type(2) > ...` selectors. Two guards protect selector quality: `_looks_autogenerated` rejects ids containing CSS-unsafe characters (`:.[]()#,` — React's `useId` emits `:r5l:`) or ending in a random-looking suffix (the `_AUTOGEN_ID` regex), and `_escape` backslash-escapes quotes before embedding text in a `:text-is("...")` selector so the text can't break the selector syntax.

Part D — Generating data & acting on it

fillable() — a filter, expressed as an explicit loop

`src/data/generator.py`

```
def fillable(elements):
    out = []
    for el in elements:
        if not el.visible or el.disabled: continue   # skip what a user couldn't use
        if el.tag in ("textarea", "select"):     out.append(el)
        elif el.widget_type == "mui_select":      out.append(el)
        elif el.tag == "input" and (el.element_type or "text") not in _SKIP_INPUT_TYPES:
            out.append(el)
    return out
```

One subtlety: `(el.element_type or "text")`. A plain `<input>` with no `type` attribute has `element_type is None`; the `or "text"` substitutes the HTML default ("text") so it isn't accidentally skipped. Buttons/hidden/file inputs are in `_SKIP_INPUT_TYPES` and fall through, returning only the genuinely fillable fields.

fill_form & the dispatcher

src/agent/executor.py

```
async def fill_form(self, values, elements):
    by_sel = {e.selector: e for e in elements}     # dict comprehension: selector -> Element
    results = []
    for selector, value in values.items():
        results.append(await self._fill_one(by_sel.get(selector), selector, value))
    return results
```

`by_sel` is a **dict comprehension** — it inverts the element list into a lookup table keyed by selector. `by_sel.get(selector)` returns the matching `Element` or `None` (the `.get` avoids a `KeyError` if the LLM ever returns a selector we don't recognize). `_fill_one` then branches on that element's shape:

```
if   element.widget_type == "mui_select":              await self._mui_select(...)
elif element.tag == "select":                          await self._select(...)
elif input checkbox/radio:                             await self.page.check(...)
elif date field:                                       fill, then press Escape
else:                                                  await self.page.fill(...)
```

The whole body is inside `try/except`; either path returns an `ActionResult` (ok / not ok with a truncated `detail`). **Failures become data, not crashes** — that's why one bad field never aborts the run, and why your FILLS log shows a row per field regardless.

_mui_select — locator mechanics up close

```
await self.page.click(selector)                                  # open popup
await self.page.wait_for_selector("[role='listbox']", state="visible")
options = self.page.locator("[role='option']")                  # lazy recipe
count = await options.count()
candidates = []
for i in range(count):
    opt = options.nth(i)                                         # i-th match, found fresh
    text = (await opt.inner_text()).strip()
    disabled = (await opt.get_attribute("aria-disabled")) == "true"
    candidates.append((i, text, disabled))
```

Each iteration builds a **tuple** `(index, text, disabled)` and collects them in a list — a plain, inspectable snapshot of the menu. Selection is then two passes: exact non-placeholder match first, else first real option. After clicking, it waits for the listbox to hide *and* the `.MuiBackdrop-root` to disappear — the backdrop is the invisible overlay that, left alive, swallowed your next click.

> **Why a tuple, not a class?** It's a throwaway grouping used three lines later. A tuple is the lightest way to carry "these three values travel together." Unpacking it reads naturally: `for i, text, disabled in candidates`.

find_submit — priority via early return

```
clickable = [e for e in elements if _is_clickable(e)]   # list comprehension = filter

for e in clickable:
    if e.in_form and e.element_type == "submit": return e     # 1 strongest
for e in clickable:
    if e.in_form and _label_matches_submit(e):   return e     # 2
in_form_buttons = [e for e in clickable if e.in_form]
if in_form_buttons:                              return in_form_buttons[-1]  # 3 last in form
for e in clickable:
    if _label_matches_submit(e):                 return e     # 4 anywhere
return None                                                    # 5 refuse to guess
```

The pattern is "**try strongest rule; `return` the instant it hits; otherwise fall to the next.**" Each rule is a small predicate (`_is_clickable` , `_label_matches_submit`) so the priority ladder reads top-to-bottom. `in_form_buttons[-1]` uses Python's negative index — the *last* button in the form, which by convention is the submit. Returning `None` rather than "first button on page" is the deliberate choice that stopped it clicking the header avatar.

Part E — Safety & observation

SafetyGate — cheap check first, LLM only when needed

`src/safety/gate.py`

```
for word in DESTRUCTIVE:
    if word in text: return SafetyVerdict(risk="destructive", ...)   # 1 free, instant
for word in AMBIGUOUS:
    if word in text: return await self._ask_llm(element)            # 2 pay for a thought
return SafetyVerdict(risk="safe", ...)                               # 3 default safe
```

This is a **tiered** design. Obvious dangers ("delete account") are caught by a substring scan — no API call, no latency. Only genuinely *ambiguous* labels cost an LLM round-trip (and that uses `SMALL_MODEL` , the cheap fast model). Everything else is assumed safe. The gate returns a `SafetyVerdict` object, and the Executor refuses to click anything marked `destructive` . The principle: **spend compute only where the cheap check can't decide.**

Observer — closures registered as event listeners

`src/agent/observer.py`

```
page.on("console", self._on_console)
page.on("pageerror", self._on_pageerror)
page.on("response", self._on_response)
```

`page.on(event, handler)` registers a callback that Playwright invokes *itself* whenever that event fires — you never call `_on_console` directly. Because the handlers are **methods bound to `self`** , each one

can append to `self.console_errors` etc. — the lists accumulate in the background for the whole session. `collect_errors` later drains and clears those lists (so a second call won't re-report the same errors), converting raw strings into typed `Finding` objects. `check_page` is the one active probe — it runs a CSS count in the browser for invalid/alert markers right now.

> **The recurring shape of this codebase:** read the world into typed objects (`Element` , `InferredContext`), make decisions on those objects in plain Python, push effects back to the browser through narrow async calls, and record every outcome as a typed result (`ActionResult` , `Finding`). Pydantic guards every boundary; `try/except` turns failures into data; `async/await` keeps the waiting cheap. Once you see that shape, every file is a variation on it.